THORRIJA

Engineering Showcase — Edición rápida (español)

Modelos computacionales deterministas reconstruidos desde código fuente. Una presentación lista para mostrar qué construí, cómo funciona y por qué importa.

Autor: Thorrija (Rubén N.)
España
Noviembre de 2026
GitHub: github.com/ThorrijaDEV

1. Introducción

Esta muestra presenta mi trabajo de ingeniería: una colección de modelos computacionales deterministas y herramientas de ingeniería que diseñé, reconstruí y probé. Cada proyecto es una reconstrucción independiente — construida a partir del código fuente, no ejecutando el original — así que todo lo que ves se calcula desde tu entrada, no está hardcodeado.

Todo funciona de forma determinista: misma entrada siempre produce la misma salida. No hay resultados generados por IA, no hay demostraciones simuladas, no hay lógica oculta. Lo que ves es lo que el código realmente hace.

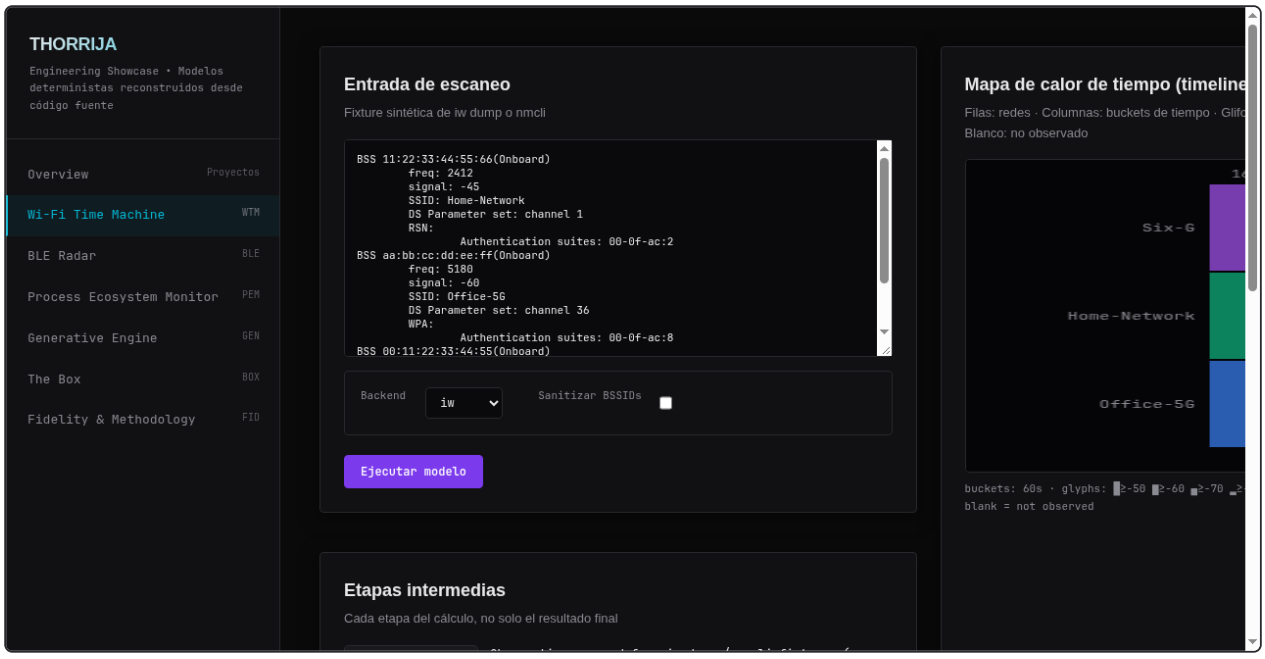
Esta edición rápida da los datos esenciales de cada proyecto: qué aborda, por qué existe, qué construí, cómo funciona y cuáles son sus limitaciones. Para detalle técnico más profundo, consulta la edición técnica.

2. Los Proyectos

A continuación se presentan todos los proyectos de la muestra, organizados por categoría. Cada uno incluye una descripción breve, el problema que aborda y su estado actual.

2.1 Wi-Fi Time Machine

Problema que aborda: No hay forma sencilla de ver cómo cambia el entorno inalámbrico con el tiempo — qué redes aparecieron, en qué canales, con qué fuerza y cuándo.
Lo que construí: Un modelo determinista que parsea volcados de escaneo Wi-Fi (formato iw o nmcli), deduplica por BSSID manteniendo la señal más fuerte, mapea frecuencias a canales 802.11 (2.4/5/6 GHz), infiere seguridad de los suites de autenticación y renderiza un mapa de calor tiempo × canales con sparkline por red.
Por qué es interesante: Convierte vuelcos de escaneo Wi-Fi en una visualización de línea de tiempo que se puede explorar, mostrando cómo cambia el entorno inalámbrico sin necesidad de hardware en vivo.
Estado: Completado — 18 pruebas, Nivel 4 de fidelidad.

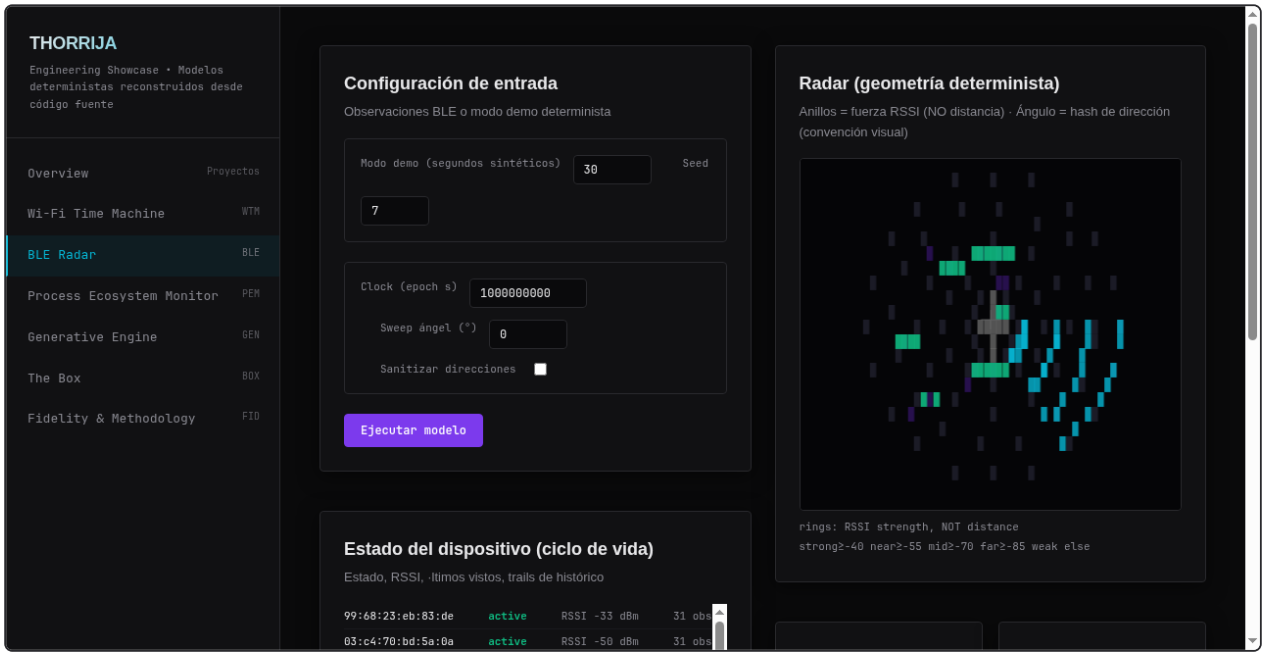


Laboratorio Wi-Fi Time Machine — entrada sintética `iw`, mapa de calor tiempo × canal y etapas intermedias del cálculo.

2.2 BLE Radar

Problema que aborda: Los dispositivos BLE cercanos emiten señales, pero no hay forma sencilla de verlos como un radar: qué dispositivos hay cerca, cuál es su fuerza de señal y cómo aparecen y desaparecen con el tiempo.
Lo que construí: Un radar determinista que simula observaciones BLE con una máquina de estado de ciclo de vida (aparecido → activo → desvaneciendo → desaparecido), suavizado EMA de RSSI (alpha=0.4), bandas de fuerza (no distancia) y un ángulo

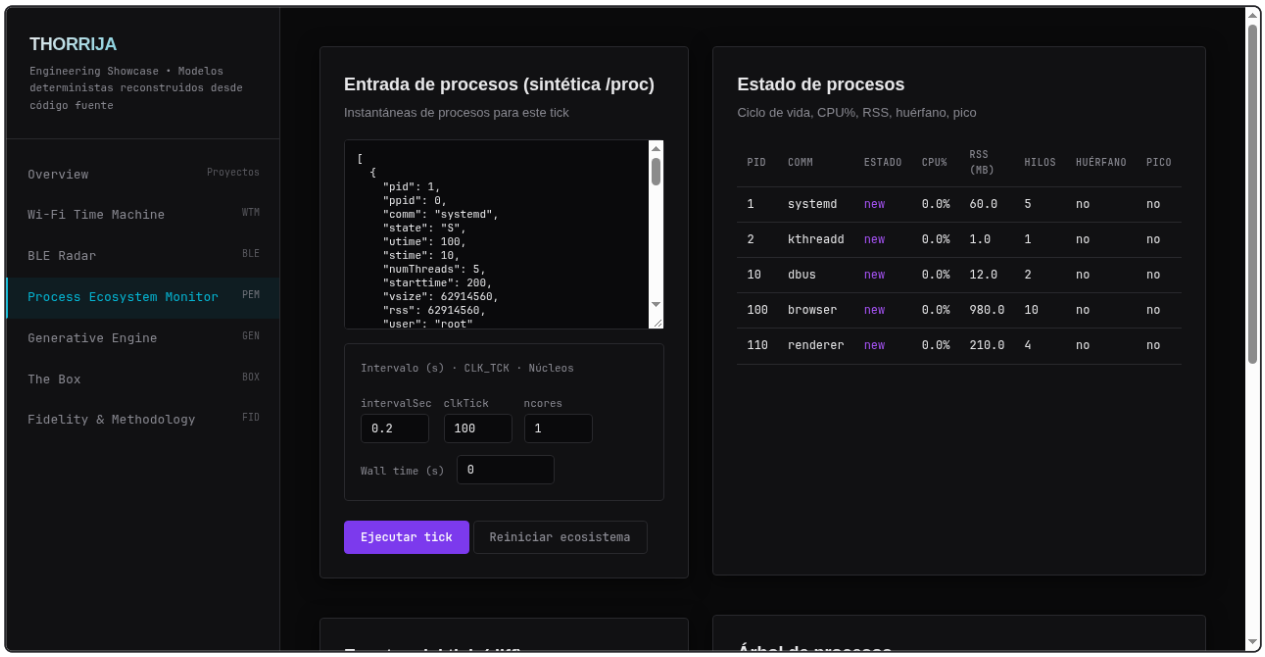
derivado del hash SHA-256 de la dirección MAC (convención visual, no orientación real). Incluye reproducción de sesiones y exportación.
Por qué es interesante: Hace visibles los dispositivos BLE cercanos como un radar determinista, con honestidad de señal: el RSSI es fuerza de señal, nunca metros físicos.
Estado: Completado — 16 pruebas, Nivel 4 de fidelidad.



Laboratorio BLE Radar — configuración determinista, geometría del radar y estado del ciclo de vida de los dispositivos.

2.3 Process Ecosystem Monitor

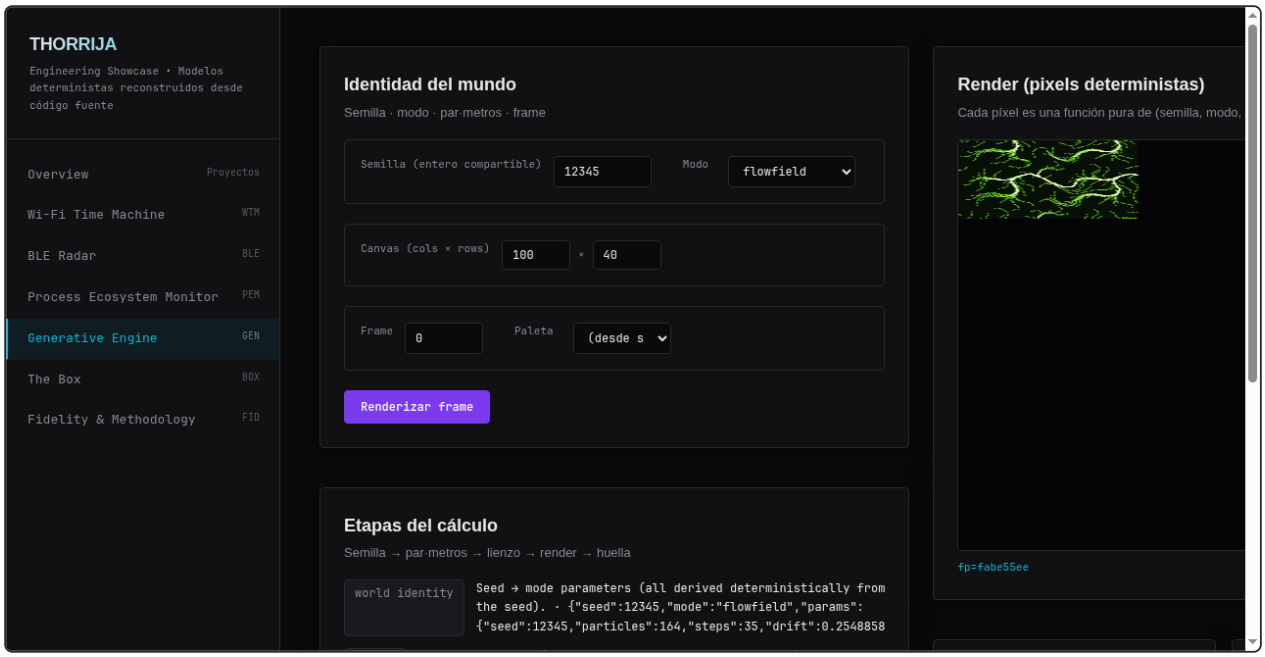
Problema que aborda: Linux /proc contiene información rica sobre procesos, pero leerla y entenderla — CPU%, árbol de procesos, reuso de PID — requiere parseo y correlación manuales.
Lo que construí: Un modelo determinista que parsea instantáneas sintéticas de estilo /proc, calcula CPU% a partir de deltas de jiffies (normalizado por núcleo), construye el árbol de procesos desde ppid, detecta reuso de PID mediante la identidad (pid, starttime), rehúerfa a huérfanos y sigue estados de ciclo de vida (nuevo → activo/inactivo → muriendo → muerto).
Por qué es interesante: Convierte instantáneas de /proc en un ecosistema de procesos comprensible con nacimientos, muertes y contabilidad de CPU — todo desde instantáneas sintéticas, sin acceder al sistema anfitrión.
Estado: Completado — 22 pruebas, Nivel 4 de fidelidad.



Laboratorio Process Ecosystem Monitor — entrada sintética de `/proc` y tabla de estado de procesos derivada.

2.4 Terminal Generative Engine

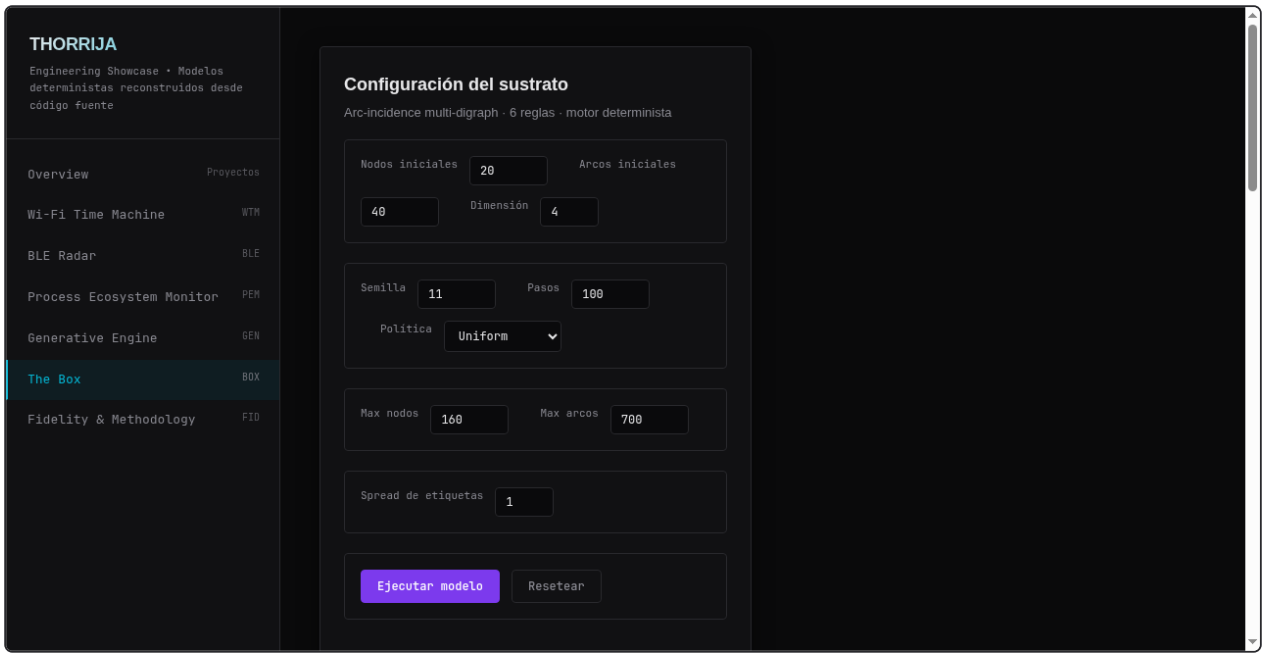
Problema que aborda: El arte generativo suele depender de estado aleatorio oculto, haciendo imposible reproducir un frame específico o compartir un resultado exacto.
Lo que construí: Un RNG splitmix64-style sin estado donde cada píxel es una función pura de (semilla, modo, frame, sal, índice). Tres modos: flowfield (campo de senos analítico), plasma (campos de color cerrados), attractor (mapa extraño de Clifford/ Dejong). Búsqueda de frame O(1) — salta a cualquier frame sin calcular los anteriores. Huella dactilar SHA-256 del mundo para compartir.
Por qué es interesante: Misma semilla + mismo frame = mismo resultado, siempre. No hay estado oculto ni aleatoriedad en tiempo de render. Comparte una semilla y cualquiera obtendrá la misma obra de arte.
Estado: Completado — 24 pruebas, Nivel 4 de fidelidad.



Laboratorio Terminal Generative Engine — controles de semilla, modo, lienzo y frame, con el render determinista.

2.5 The Box

Problema que aborda: La mayoría de los grafos almacenan identidad y linaje como campos explícitos. The Box pregunta: ¿qué pasaría si la identidad y el linaje no se almacenan en absoluto, sino que emergen de reglas de reescritura?
Lo que construí: Un sustrato relacional arc-incidencia multi-digraph donde nodos y arcos comparten un mismo espacio de ids, los arcos pueden tener otros arcos como extremos, y seis reglas de reescritura (GATHER, BOND, TAG_PROP, QUIET, DESCEND, FUSE) modifican la estructura. La identidad (quién se fusionó con quién) y el linaje (quién engendró a quién) son propiedades derivadas de las etiquetas de los arcos, no campos almacenados en el núcleo. Cuatro políticas del motor, métricas estructurales (grados, Gini, componentes, beta0/beta1, modularidad), 14 observables nombrados, modelos nulos y p-valores empíricos.
Por qué es interesante: Demuestra cómo propiedades complejas como identidad y linaje pueden emerger de reglas locales simples sin ser almacenadas explícitamente — una forma diferente de pensar la computación sobre grafos.
Estado: Completado — 102 pruebas (42+25+12+23), Nivel 3 de fidelidad (BLAKE2b puro TS, no byte-compatible con Python hashlib.blake2b).



Laboratorio The Box — configuración del sustrato arc-incidencia: nodos y arcos, dimensión, semilla, pasos, política del motor y spread de etiquetas.

2.6 Sigscan

Problema que aborda: El análisis forense necesita detectar y extraer formatos de archivo embebidos (JPEG, PNG, ELF) dentro de volcado binarios sin conocer sus límites.
Lo que construí: Un escáner forense de firmas mágicas en Go: detecta JPEG/PNG/ELF embebidos y puede tallarlos a disco. Solo stdlib, binario estático.
Estado: Completado.

2.7 Dnsdrill

Problema que aborda: Entender DNS requiere ver la estructura real de los paquetes, no solo las respuestas resueltas.
Lo que construí: Un cliente DNS que construye paquetes RFC 1035 a mano con struct: punteros de compresión, fallback TCP en truncados, 7 tipos de registro. Una vista transparente del protocolo DNS en el cable.
Estado: Completado.

2.8 Silicon

Problema que aborda: Entender la arquitectura de CPU requiere construir una desde cero.
Lo que construí: Una CPU de 8 bits construida desde cero en Rust: ISA propia, ensamblador, desensamblador, VM determinista, pantalla virtual de 32×32. Cero dependencias.
Estado: Completado.

2.9 Forja

Problema que aborda: Los compiladores suelen ser cajas negras; entenderlos requiere construir uno desde cero.
Lo que construí: Un compilador en Rust que traduce un lenguaje tipo u8 a ensamblador SILICON: variables en memoria, ALU sobre el acumulador, saltos por banderas, print_num con división por restas y pruebas end-to-end sobre la VM real.
Estado: Completado.

2.10 Polea

Problema que aborda: Los sistemas de mensajería pub/sub suelen ser complejos con muchas dependencias; entenderlos requiere construir uno desde cero.
Lo que construí: Un broker de mensajería pub/sub en C++17 sin dependencias: protocolo binario propio con cabecera de 13 bytes, delimitación por longitud y CRC-32, reenvío ordenado con un solo hilo sobre poll(2), colas acotadas con backpressure y cliente CLI para publicar y escuchar temas.
Estado: Completado.

2.11 Cask

Problema que aborda: Los almacenes direccionables por contenido necesitan deduplicación y recuperación ante corrupción.
Lo que construí: Un almacén solo-append direccionable por contenido escrito en C puro: formato binario propio con CRC32 + SHA-256, deduplicación por contenido y recuperación automática ante corrupción o truncamiento.
Estado: Completado.

2.12 Neondrift

Problema que aborda: Los juegos arcade suelen requerir muchos activos; construir uno desde cero demuestra programación de juegos.
Lo que construí: Un endless runner arcade en un solo HTML: física y colisiones propias, chiptune generado con WebAudio (cero assets), partículas y récord local.
Estado: Completado.

2.13 Clickwheel + Console

Problema que aborda: Los reproductores de música y las consolas web suelen ser experiencias separadas; combinarlas muestra diseño de interfaz multiplataforma.
Lo que construí: Dos reproductores, una idea: música que se opera. App Android estilo iPod con click-wheel háptico (Compose) que reproduce música local y busca en YouTube Music; y una consola web hi-fi vintage sin dependencias.
Estado: Completado.

2.14 Syspeek

Problema que aborda: Los paneles de monitorización de sistemas suelen requerir muchas dependencias; una TUI en curses muestra lo que se puede hacer con dependencias mínimas.
Lo que construí: Panel TUI en curses puro — cero dependencias. CPU por núcleo, memoria, tasas de red y procesos principales leídos directamente de /proc con parsers hechos a mano.
Estado: Completado.

2.15 Hackdeck

Problema que aborda: Las utilidades de pentest suelen estar dispersas en muchas herramientas; un mazo offline en un solo HTML las hace accesibles desde cualquier navegador.
Lo que construí: Mazo offline de utilidades pentest en HTML único (~15 KB): base64, hex, JWT, SHA-1/256/512, XOR, César, UUID. Cero red, cero telemetría — todo corre en tu navegador.
Estado: Completado.

2.16 Canarytrap

Problema que aborda: Detectar accesos no autorizados requiere tokens canary que alerten cuando alguien los toca.
Lo que construí: Canary tokens autoalojados: claves AWS falsas, .env señuelo y web bugs que disparan un webhook cuando alguien los toca. Solo stdlib, tus registros nunca salen de tu VPS.
Estado: Completado.

2.17 Loglens

Problema que aborda: Detectar amenazas en registros requiere un SIEM ligero que funcione sin infraestructura pesada.
Lo que construí: SIEM de bolsillo: detecta fuerza bruta SSH, enumeración de usuarios, escaneos web y picos de error en tus registros — cero dependencias, 100% offline, informe HTML.
Estado: Completado.

2.18 Reconkit

Problema que aborda: El reconocimiento de activos autorizados requiere un multitool que combine subdominios, escaneo de puertos y dir busting.
Lo que construí: Multitool de reconocimiento para activos autorizados: subdominios vía CT logs, escáner de puertos asíncrono y dir busting — todo en sesiones SQLite con informe HTML unificado.

Estado: Completado.

2.19 Port Authority

Problema que aborda: Saber qué proceso escuchó en qué puerto y cuándo requiere seguimiento histórico.
Lo que construí: Un historiador de puertos de red para Linux: registra quién escuchó dónde y cuándo en una bitácora SQLite local.
Estado: Completado.

2.20 Otros trabajos y experimentos

Red Neuronal Modular: Arquitectura de red neuronal construida desde cero en Python con un colaborador, extensible mediante plugins.
Sistema de IA personal: En desarrollo: automatización de tareas digitales y flujos de trabajo propios.
Automatizaciones personales: Scripts y herramientas pequeñas que resuelven fricciones diarias en mi escritorio Linux.
Edición de vídeo: Producciones con más de 20.000 visualizaciones entre CapCut y DaVinci Resolve.
Experiencia OSINT: Exploración de herramientas de ciberseguridad en entornos controlados.

3. Cómo demostrar

Cada sección de proyectos anteriores explica qué mostrar. Para los laboratorios interactivos (WTM, BLE, PEM, GenArt, The Box), abre la aplicación web, selecciona el laboratorio desde la barra lateral, ajusta los parámetros y haz clic en ejecutar. Todos los modelos son deterministas — misma semilla y entrada siempre producen el mismo resultado.

Para los proyectos estáticos (Sigscan, Dnsdrill, Silicon, etc.), consulta sus respectivos repositorios de GitHub para el código fuente y uso.

4. Contacto y repositorios

- **Autor:** Thorrija (Rubén N.)
- **Ubicación:** España
- **GitHub:** github.com/ThorrijaDEV
- **Showcase:** github.com/ThorrijaDEV/showcase
- **Modelos:** github.com/ThorrijaDEV/models
- **Portafolio:** github.com/ThorrijaDEV/Portafolio

Todos los modelos son deterministas: misma entrada → misma salida. No requieren hardware, ni claves de API, ni servicios externos para las demostraciones. Todos los datos mostrados son sintéticos o calculados desde tu entrada — nada está fabricado.

PYEOF